

PFA — MLOps 2025-2026 | ENSIAS

Industrialisation d'un Système de Détection de Pathologies Dentaires

Une Approche MLOps de Bout en Bout

Encadrant :
Mr. Mohamed Lazzar

Falaq Jad

Jury :
Pr. Abdellatif El Afia
Pr. Reddouane Chiheb

Plan de Présentation

01

Introduction & Contexte

Problème clinique, enjeu MLOps et architecture globale du système

02

Données et Préparation

Audit du corpus, restructuration des classes et pipeline DVC

03

Modélisation

YOLOv8, suivi MLflow et sélection du modèle Champion

04

Déploiement & Industrialisation

API REST, Docker, CI/CD et infrastructure cloud

05

Bilan & Perspectives

Réalisations MLOps, limites et évolutions futures

01

Introduction & Contexte

Du problème clinique à la solution MLOps — pourquoi
l'industrialisation compte



Contexte Clinique et Enjeu MLOps

Le problème : 3,5 milliards de personnes touchées par les maladies bucco-dentaires dans le monde. L'interprétation des radiographies panoramiques est **longue, exigeante et sujette à erreurs**.

L'idée : un **second regard automatique** qui analyse la radiographie en quelques secondes et signale les zones à examiner, sans remplacer le dentiste.

Le vrai défi : la plupart des projets IA médicale fonctionnent dans un notebook mais **ne passent jamais en production**.

3,5 Md

personnes touchées par les
maladies bucco-dentaires

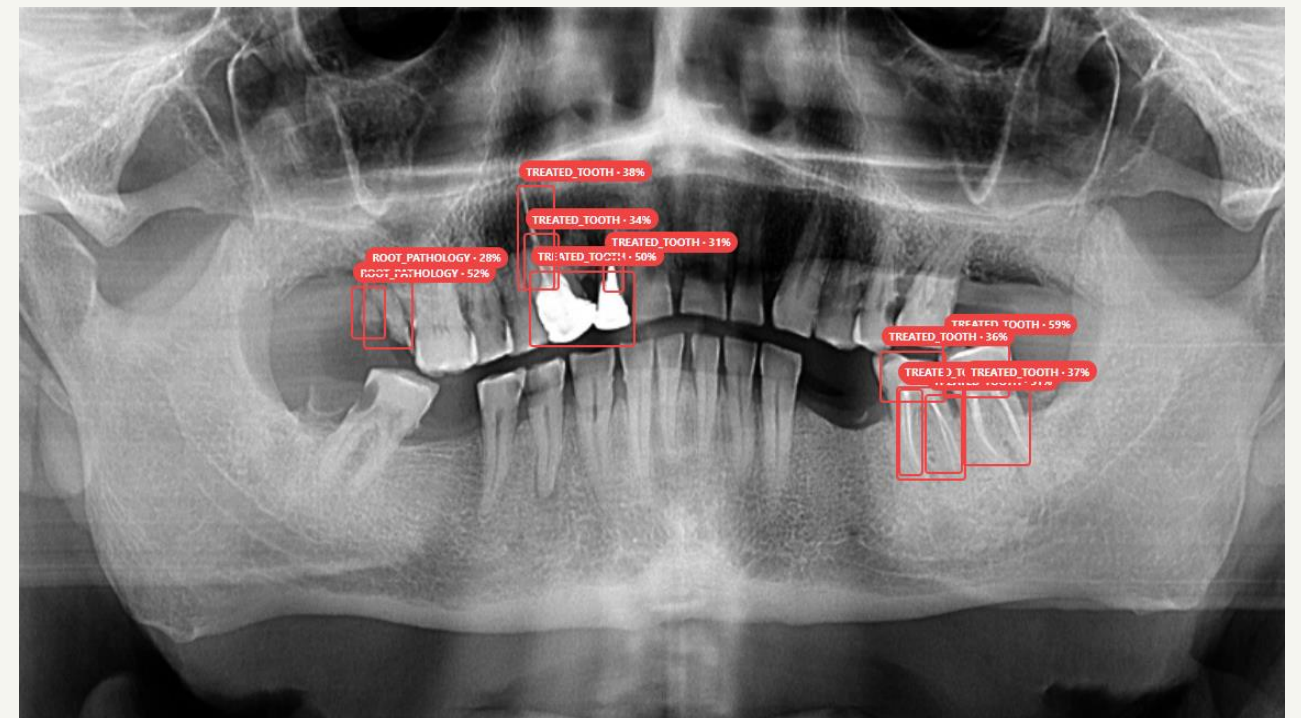
7

catégories d'anomalies détectées

< 5s

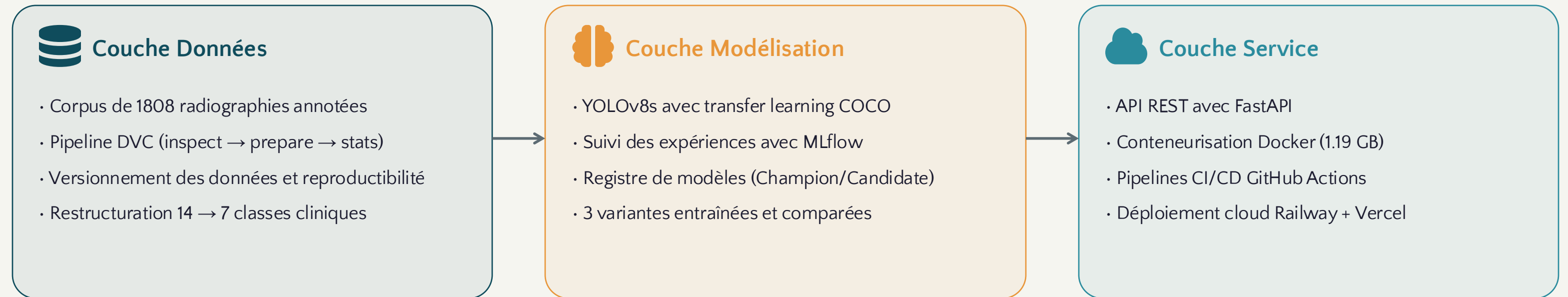
temps d'analyse par radiographie

"Ce projet répond à cette question en adoptant les pratiques du MLOps, une discipline qui réconcilie la rigueur de l'ingénierie logicielle avec les exigences des systèmes d'apprentissage automatique."



Architecture Globale du Système

Trois couches indépendantes : Données → Modélisation → Service



Séparation des responsabilités — Un changement dans une couche n'affecte pas les autres. On peut améliorer le modèle sans toucher au service, modifier l'interface sans toucher aux données. Les pipelines CI/CD détectent automatiquement les pannes.

Stack Technologique

Python 3.11 | YOLOv8s (Ultralytics) | DVC 3.x | MLflow 2.x | FastAPI + Uvicorn | Docker | GitHub Actions | React 19 + Vite | Railway | Vercel | pytest

02

Données et Préparation

Audit, restructuration et versionnement d'un corpus
clinique de 1808 radiographies



Audit du Dataset et Restructuration

Résultat de l'audit structurel

Partition	Images	Labels	Manquant s	Corrompu s	Invalides
train	1375	1375	0	0	2
eval	153	153	0	0	0
external	180	180	0	0	1

Résultat — Dataset structurellement propre. Seules 3 annotations invalides (bbox width = 0) filtrées sans impact.

Restructuration 14 → 7 classes

Classes originales		Classe finale
Carious lesion	→	CARIES
Apical periodontitis	→	PERIAPICAL_PATHOLOGY
Bone resorption, Furcation	→	PERIODONTAL_BONE
Impacted tooth	→	IMPACTED_TOOTH
Root fragment, Root resorption	→	ROOT_PATHOLOGY
Prosthetic, Obturation, Endodontic	→	TREATED_TOOTH
Implant, Orthodontic, Surgical	→	DEVICE_IMPLANT

1808

images conservées

24 888

annotations propres

7

classes cliniques

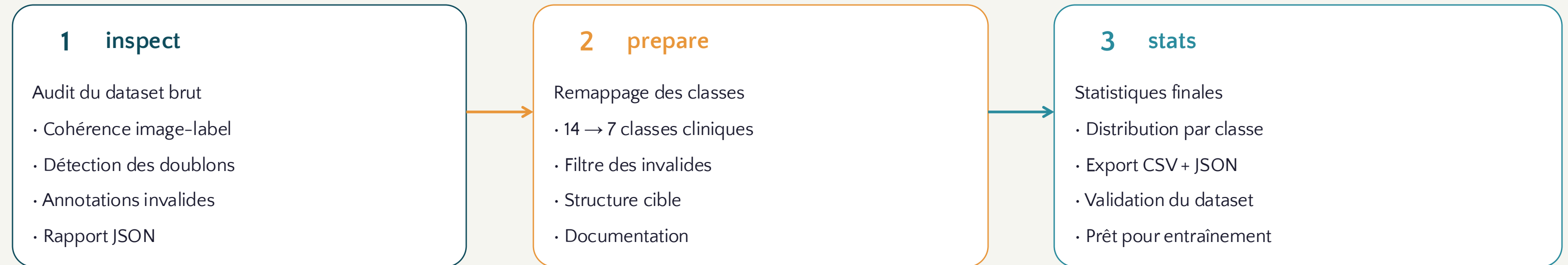
0

erreur structurelle

13.5

objets/radio en moyenne

Pipeline DVC – Reproductibilité des Données



\$ dvc repro — une seule commande pour reproduire l'intégralité de la chaîne de transformation

Dataset brut vs Dataset traité

Version	Images	Annotations	Classes	Invalides	Statut
Brut	1808	24 891	14	3	Déséquilibré
Traité	1808	24 888	7	0	Équilibré

Avantage clé — DVC détecte automatiquement les étapes à réexécuter selon les modifications. N'importe qui partant du même dataset brut obtient **exactement le même dataset traité**.

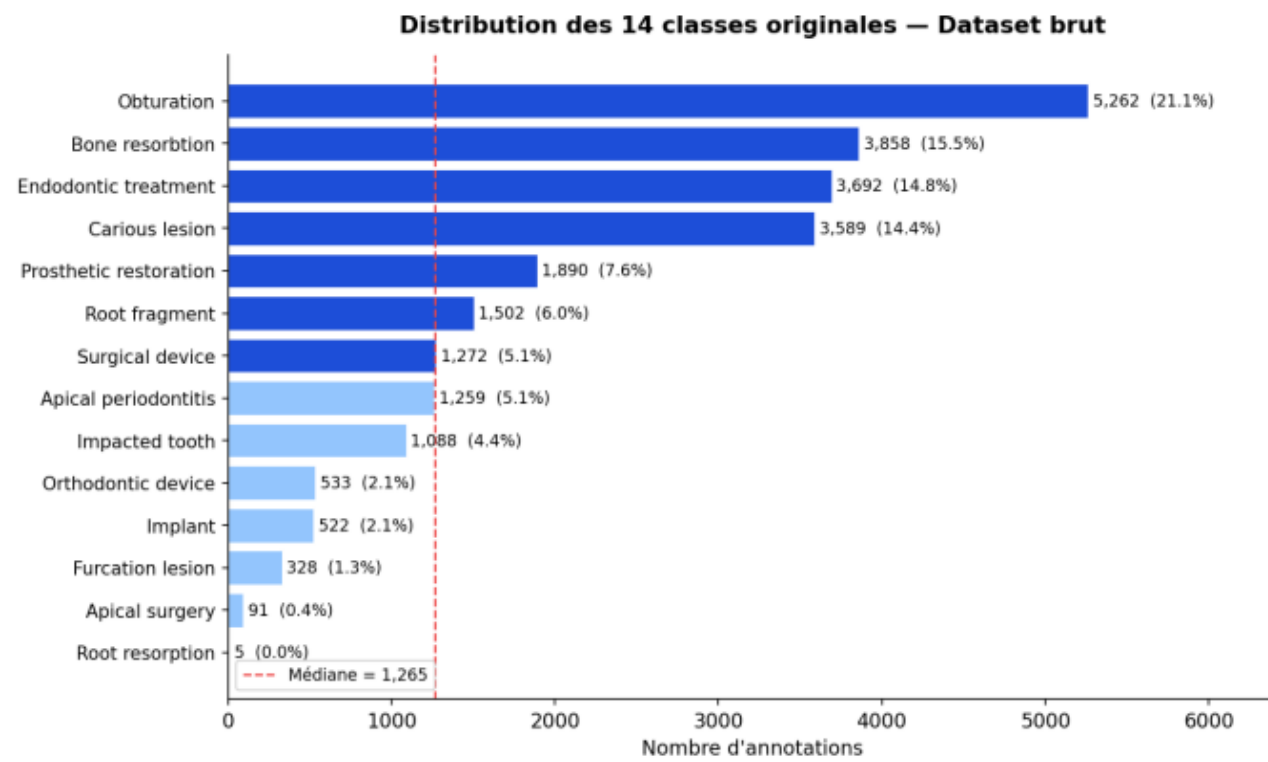


FIGURE 3.1 – Distribution des 14 classes originales — déséquilibre sévère entre les classes

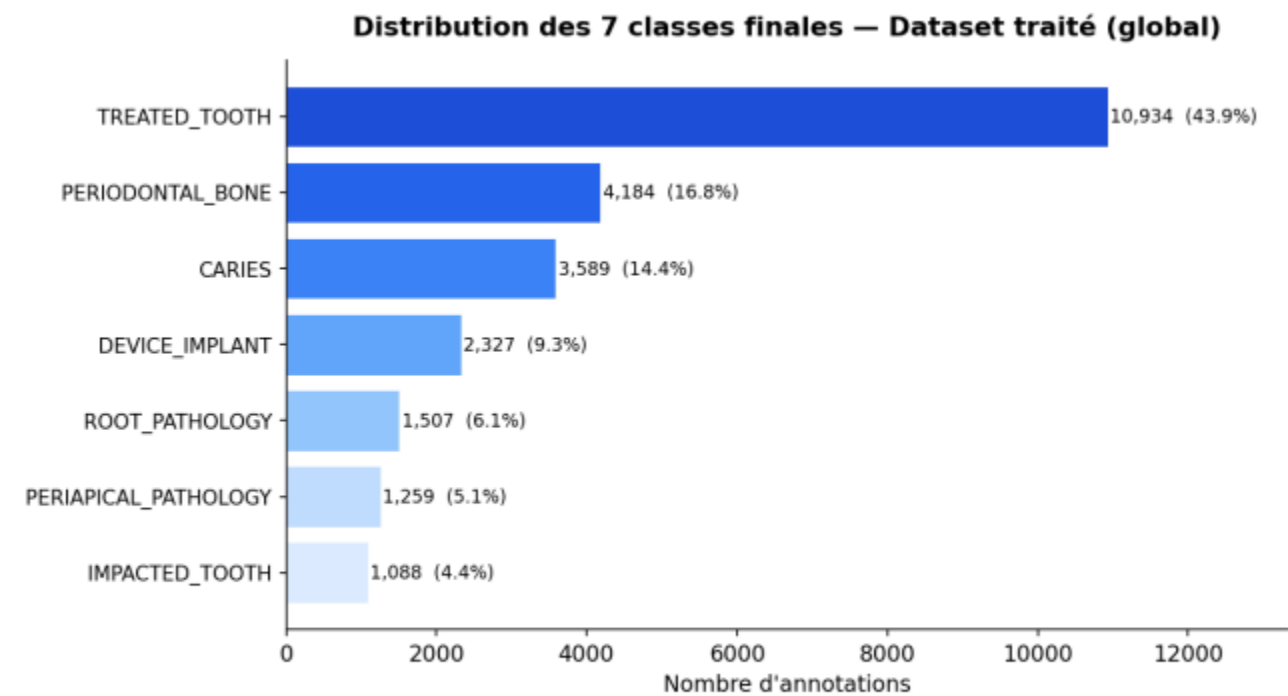


FIGURE 3.2 – Distribution après remappage — 7 classes plus équilibrées

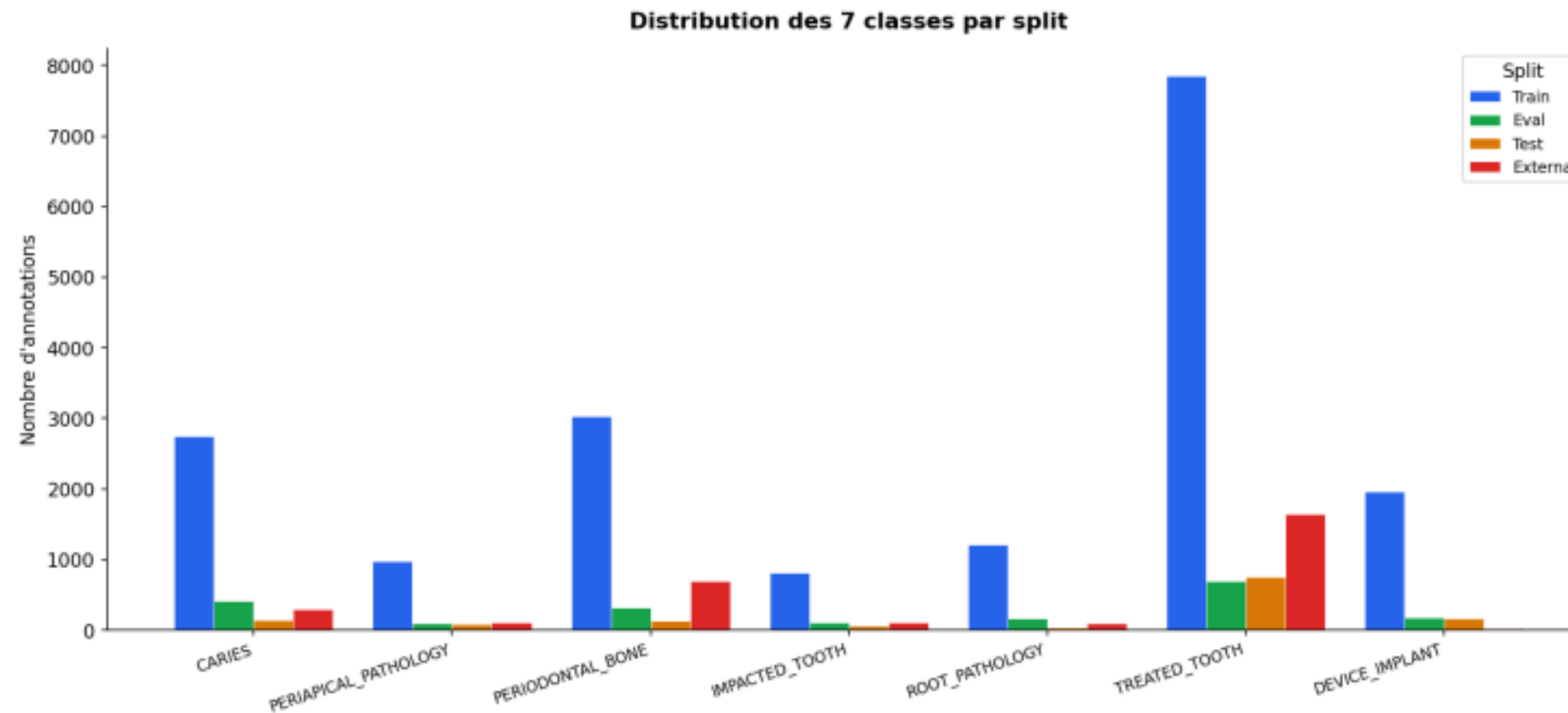


FIGURE 3.4 – Distribution des 7 classes par partition dans le dataset traité

03

Modélisation et Expérimentation

YOLOv8, traçabilité MLflow et sélection du modèle
Champion



Configuration et Suivi des Expériences

Configuration d'entraînement

Paramètre	Valeur	Justification
Architecture	YOLOv8s	11.2M params, bon compromis
Résolution	832 px	Préserve les petites lésions
Optimiseur	AdamW	Meilleure convergence
Learning rate	0.0005	Faible (pré-entraîné)
Patience	35 epochs	Arrêt anticipé
Flip horizontal	0.5	Seule augmentation
Flip vertical	0.0	Désactivé (non réaliste)

Traçabilité avec MLflow

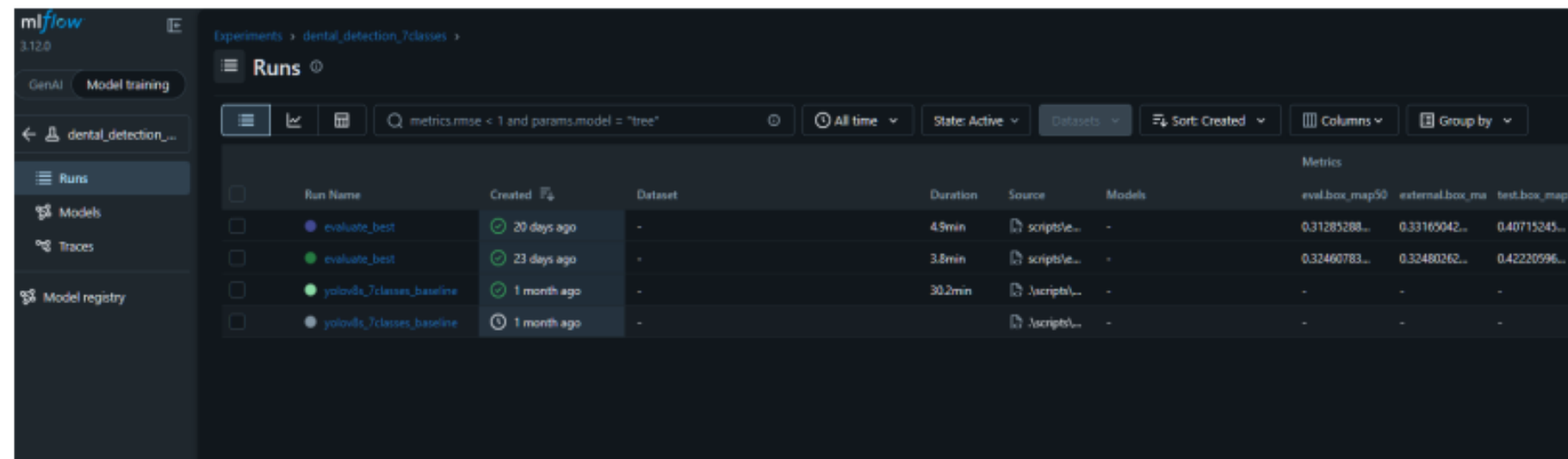
Chaque entraînement ouvre un **run MLflow** qui consigne automatiquement :

- **74 paramètres** traçables (seed, chemins, classes)
- Métriques d'évaluation par partition (eval/test/external)
- Artéfacts : checkpoints, configurations, courbes



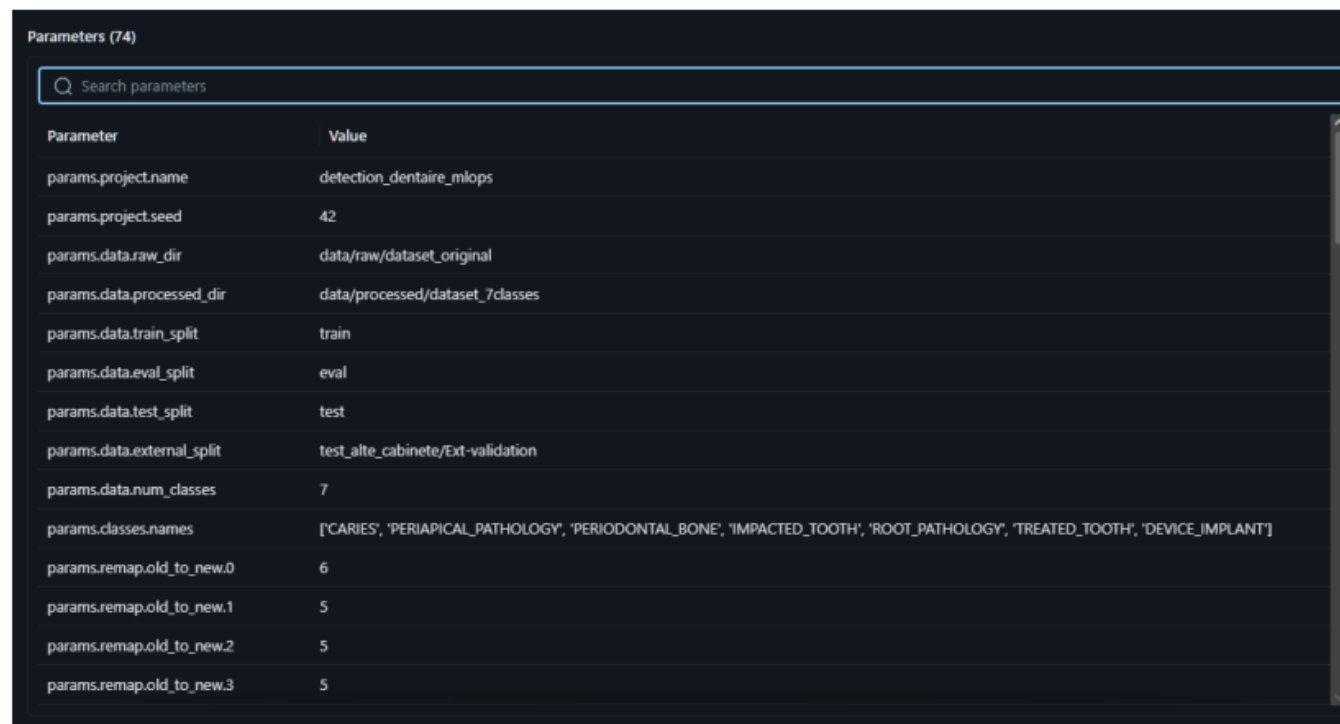
Choix médicalement cohérent — En imagerie médicale, les augmentations agressives (rotations, distorsions) introduisent des artefacts irréalistes. Seul le flip horizontal est utilisé, car une radiographie à l'envers n'a pas de sens clinique.

Configuration et Suivi des Expériences



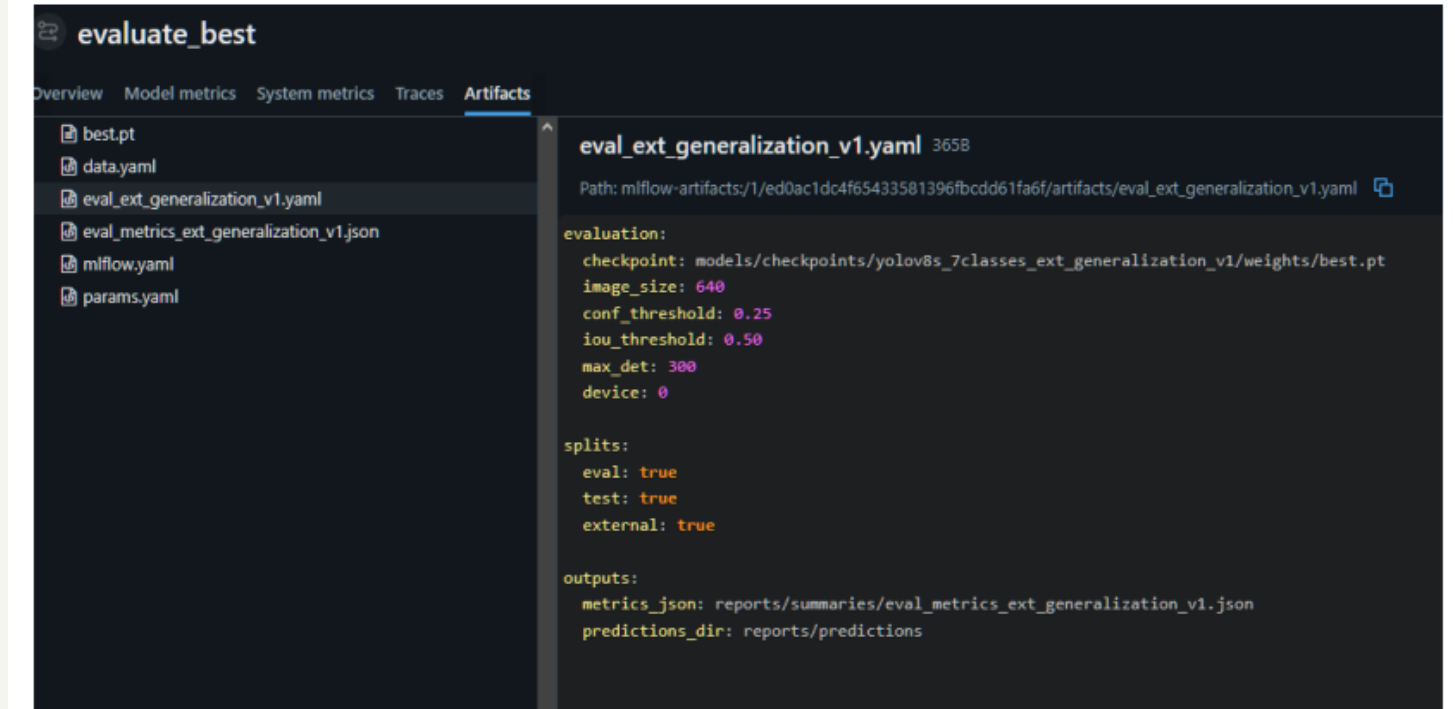
Run Name	Created	Dataset	Duration	Source	Models	Metrics
evaluate_best	20 days ago	-	49min	scriptable...	-	eval_box_map50: 0.31285288...
evaluate_best	23 days ago	-	3.8min	scriptable...	-	eval_box_map50: 0.32460783...
yolov8s_7classes_baseline	1 month ago	-	30.2min	scriptable...	-	eval_box_map50: -
yolov8s_7classes_baseline	1 month ago	-	-	scriptable...	-	eval_box_map50: -

FIGURE 4.1 – Interface MLflow — liste des runs avec métriques comparées



Parameter	Value
params.project.name	detection_dentaire_mlops
params.project.seed	42
params.data.raw_dir	data/raw/dataset_original
params.data.processed_dir	data/processed/dataset_7classes
params.data.train_split	train
params.data.eval_split	eval
params.data.test_split	test
params.data.external_split	test_alte_cabinete/Ext-validation
params.data.num_classes	7
params.classes.names	['CARIES', 'PERIAPICAL_PATHOLOGY', 'PERIODONTAL_BONE', 'IMPACTED_TOOTH', 'ROOT_PATHOLOGY', 'TREATED_TOOTH', 'DEVICE_IMPLANT']
params.remap.old_to_new.0	6
params.remap.old_to_new.1	5
params.remap.old_to_new.2	5
params.remap.old_to_new.3	5

FIGURE 4.2 – Détail d'un run MLflow — 74 paramètres traçables



Artifact	Path
best.pt	models/checkpoints/yolov8s_7classes_ext_generalization_v1/weights/best.pt
data.yaml	data/processed/dataset_7classes
eval_ext_generalization_v1.yaml	reports/summaries/eval_ext_generalization_v1.yaml
eval_metrics_ext_generalization_v1.json	reports/summaries/eval_metrics_ext_generalization_v1.json
mlflow.yaml	mlflow-artifacts/1/ed0ac1dc4f65433581396fbcd61fa6f/artifacts/eval_ext_generalization_v1.yaml
params.yaml	reports/summaries/params.yaml

FIGURE 4.3 – Artéfacts loggés pour un run — checkpoint, configurations et métriques

Résultats et Sélection du Modèle Champion

Comparaison des variantes

TABLE 4.2 – Comparaison des deux variantes principales sur les trois partitions

Modèle	Partition	Précision	Rappel	mAP50	mAP50-95
Baseline	eval	0.695	0.393	0.325	0.136
Baseline	test	0.860	0.467	0.422	0.211
Baseline	external	0.682	0.383	0.325	0.146
Candidate (ext.)	eval	0.689	0.368	0.313	0.138
Candidate (ext.)	test	0.894	0.441	0.407	0.199
Candidate (ext.)	external	0.757	0.383	0.332	0.160

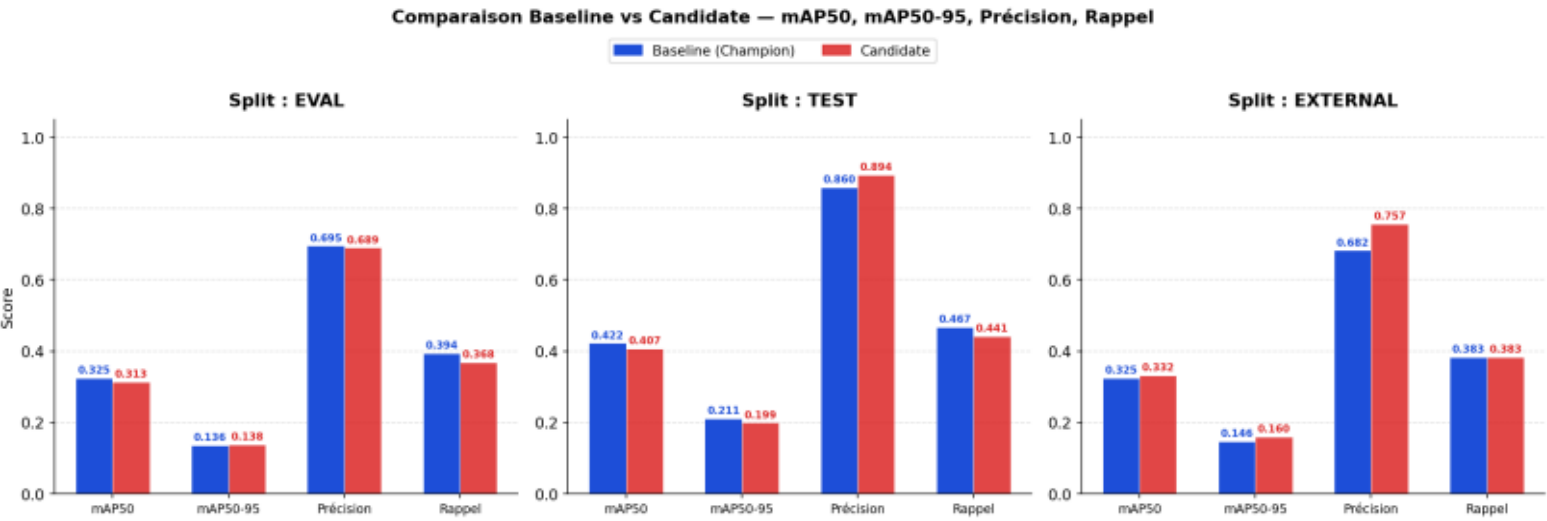


FIGURE 4.4 – Comparaison visuelle Baseline vs Candidate sur les trois partitions

Registre de modèles

Rôle	Modèle	Justification
Champion	yolov8s_7classes_baseline	Meilleur compromis global eval + test
Candidate	yolov8s_7classes_ext_gen_v1	Meilleur sur external, moins bon globalement

Pourquoi Baseline ?

- Meilleurs scores sur **test et eval**
- Rappel supérieur — manquer une anomalie est plus grave qu'un faux positif
- Promotion via **CLI** sans modifier le code du service
- Alias stable pointé par tous les composants

Résultats et Sélection du Modèle Champion

Comparaison des variantes

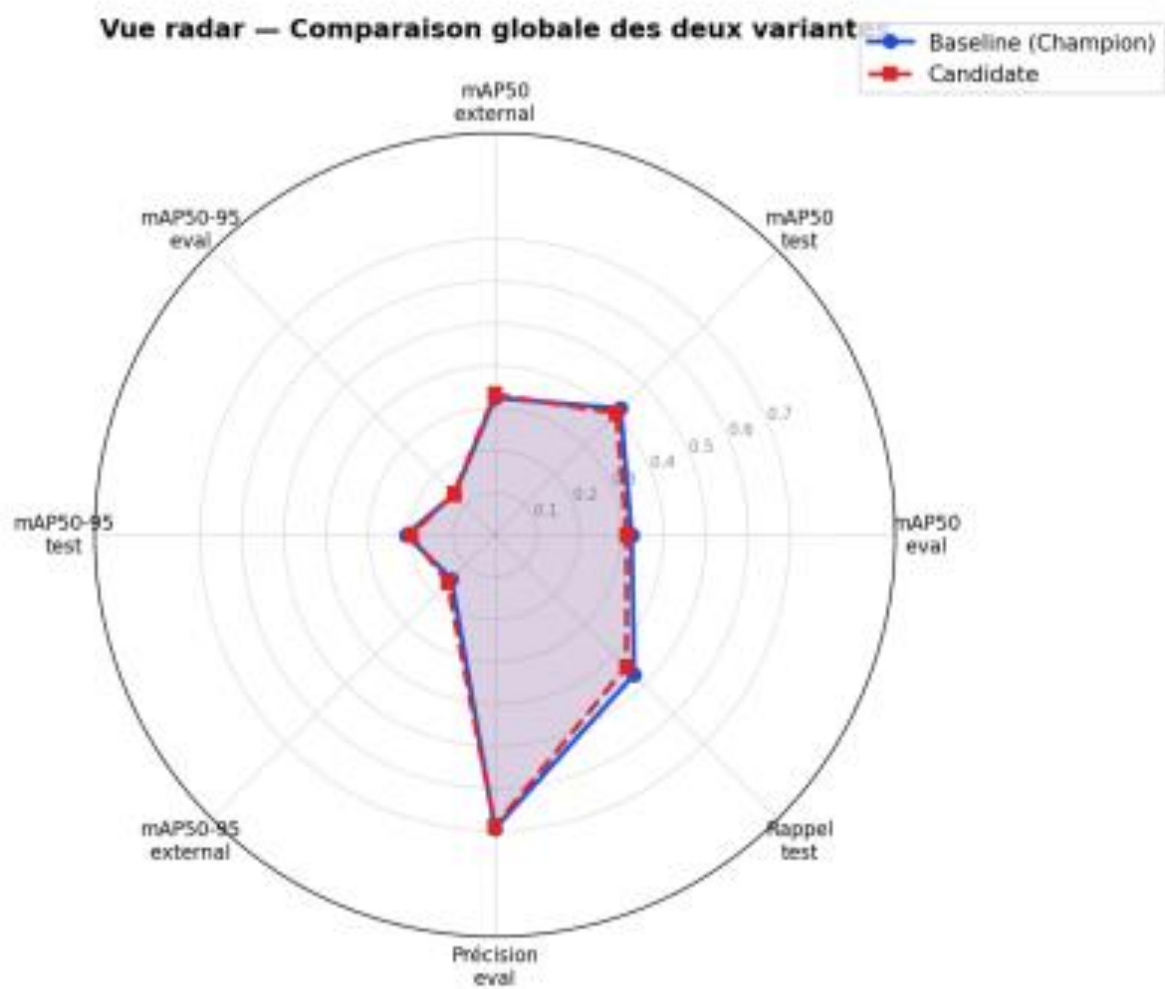


FIGURE 4.5 – Vue radar synthétique — comparaison globale des deux variantes

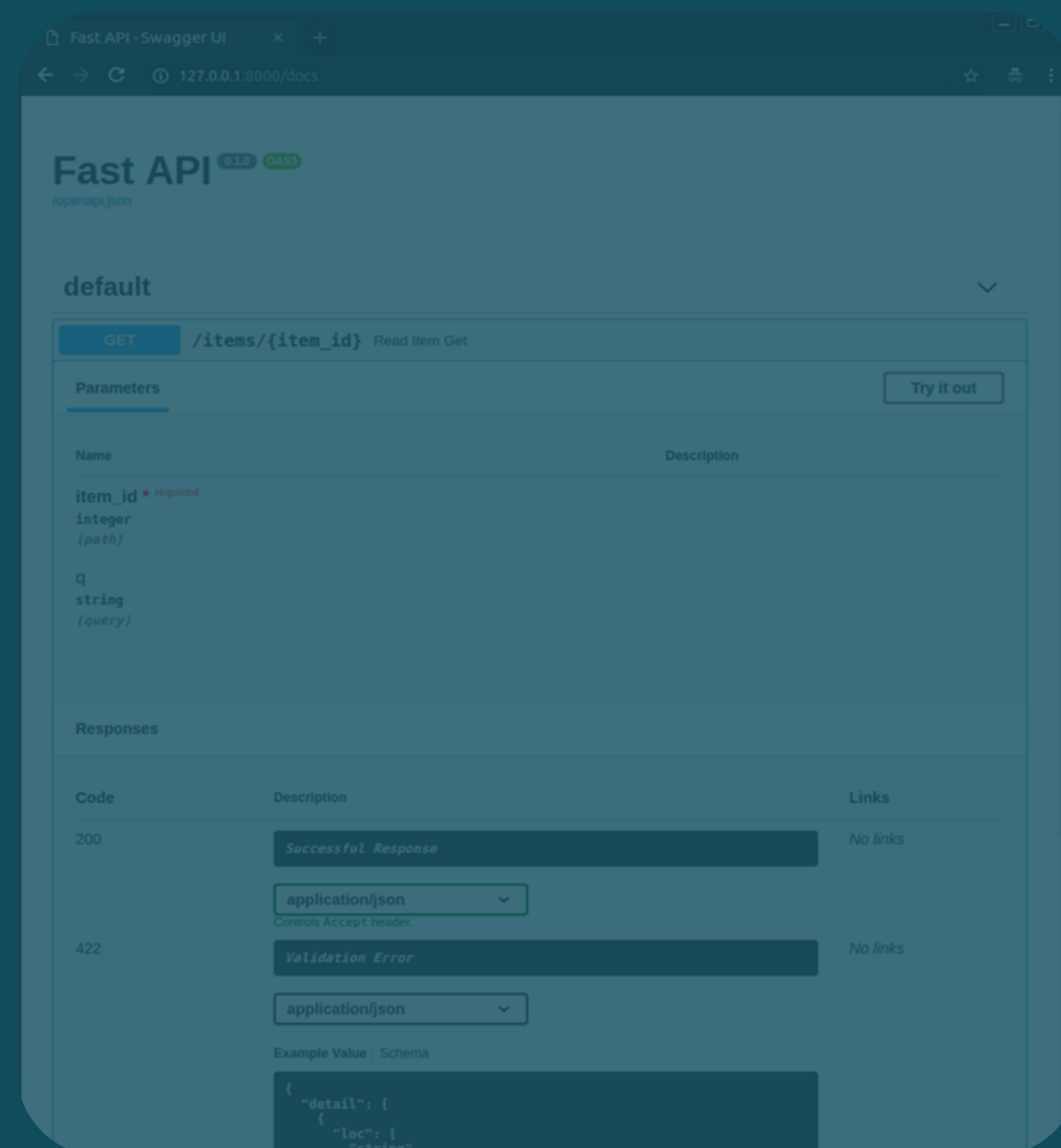
TABLE 4.3 – État du registre de modèles

Rôle	Modèle	Justification
Champion	yolov8s_7classes_baseline	Meilleur compromis global eval + test
Candidate	yolov8s_7classes_ext_gen_v1	Meilleur sur external, moins bon globalement
Archived	yolov8s_7classes_v2_small	Expérimentation petites lésions, archivé

04

Déploiement et Industrialisation

De l'expérience au service cloud opérationnel — API,
Docker, CI/CD



API REST et Interface Utilisateur

Endpoints de l'API FastAPI

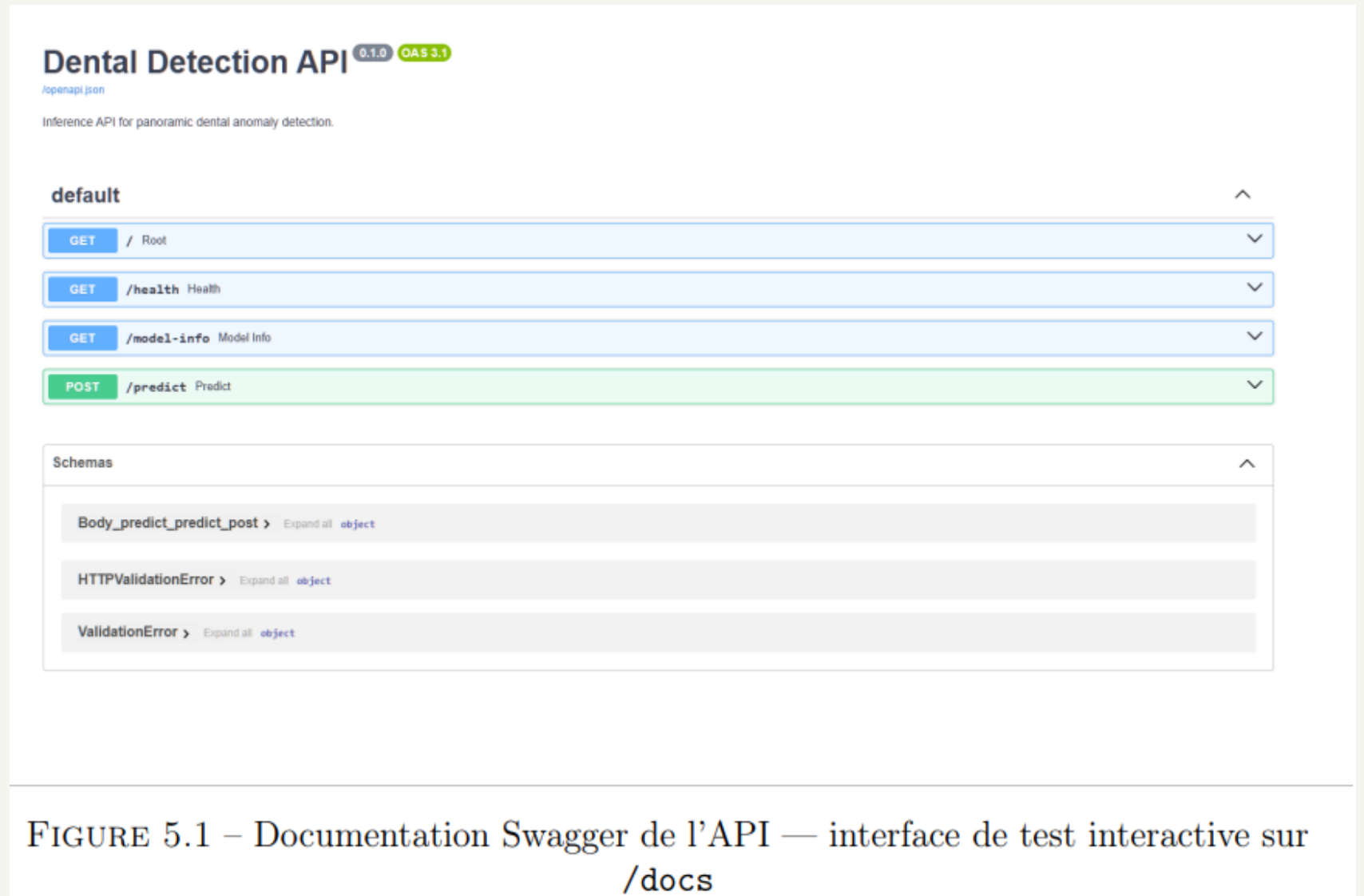


FIGURE 5.1 – Documentation Swagger de l'API — interface de test interactive sur /docs

Lazy loading — Le modèle n'est chargé qu'à la première requête d'inférence. Le service répond immédiatement aux requêtes /health et /model-info.

1. Upload
Radiographie PNG/JPG



2. Préprocessing
Canvas côté client



3. Inférence
API + YOLOv8



4. Visualisation
Bounding boxes + scores

API REST et Interface Utilisateur

Interface React 19 + Vite

DENTAL AI AGENT

Analyse intelligente de radiographies panoramiques dentaires

Upload une radio, ajuste les controles d analyse et laisse le modele Champion detecter les anomalies dentaires avec un rendu visuel des bbox directement sur l image.

Modele actif

champion

Checkpoint

Disponible

API

https://detection...

Upload et controles

Prepare la radio puis configure l analyse du modele.

+

Glisse ta radiographie ici ou clique pour uploader

Formats acceptes: JPG, PNG, BMP, TIFF, WEBP.
Le site reconstruit une image preprocessee avant de l envoyer au modele.

Taille cible d analyse

896 px

Confiance minimale

0.25

Seuil IoU

0.50

Nombre max de detections

300

Luminosite preprocessing

100%

Contraste preprocessing

100%

Lancer l analyse

Reinitialiser

Fichier selectionne

310.jpg

Originale: 2652 x 1536 -> preprocessee: 896 x 519

Rendu visuel avec bbox

La radio preprocessee est celle qui est envoyee au modele Champion.

5/17/2023 7:32 AM - STD PANORAMIC - 66kV 6mA 14.6s - 7.20uGym2

Predictions et synthese

Lecture rapide des detections renvoyees par le modele.

Detections

10

Resolution envoyee

896 x 519

Reshape applique

x0.34

Reglage confiance

0.25

Repartition par classe

TREATED_TOOTH

4

DEVICE_IMPLANT

6

Liste detaillee

TREATED_TOOTH

548, 210, 570, 266

60%

DEVICE_IMPLANT

579, 156, 649, 244

57%

TREATED_TOOTH

339, 224, 358, 277

55%

TREATED_TOOTH

273, 213, 307, 263

50%

DEVICE_IMPLANT

342, 81, 405, 200

48%

DEVICE_IMPLANT

220, 168, 308, 231

48%

TREATED_TOOTH

274, 220, 289, 260

45%

DEVICE_IMPLANT

481, 96, 547, 211

43%

DEVICE_IMPLANT

180, 402, 293, 451

43%

ENSIAS — MLOps 2025-2026

13

Docker et Pipelines CI/CD

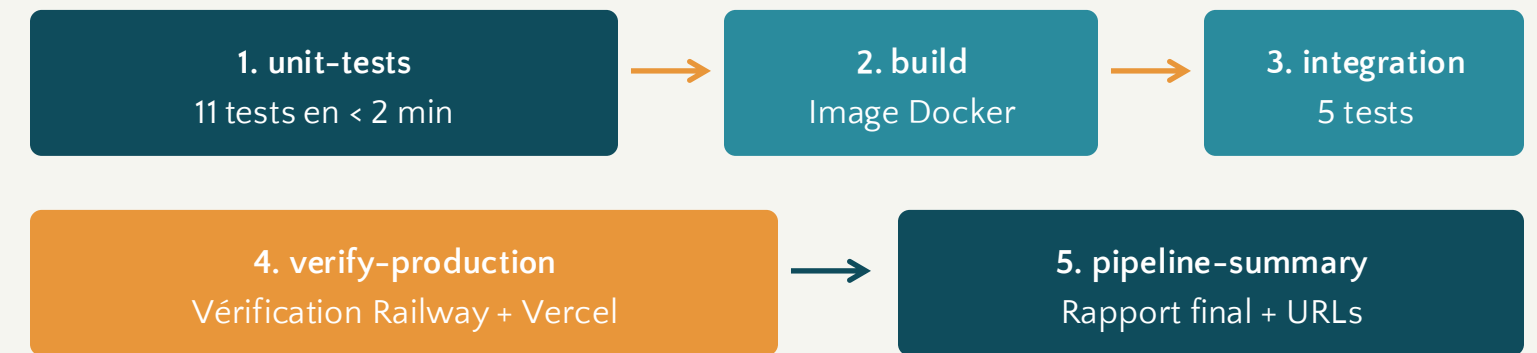
Image Docker optimisée

Image	Taille	Contenu	Usage
cpu (ancienne)	3.74 GB	PyTorch CUDA	Développement GPU
latest (optimisée)	1.19 GB	CPU + checkpoint	Production, auto-suffisante

Le checkpoint Champion (best.pt, 21.5 MB) est **intégré dans l'image**. Une commande suffit pour lancer le service complet sur n'importe quelle machine :

```
docker run -p 8000:8000 dental-detection-api:latest
```

Pipeline CD – 5 jobs séquentiels



Valeur des tests d'intégration — Ils ont détecté des problèmes invisibles autrement : séparateur de chemin Windows/Linux, conflit device=0 sans CUDA, dépendance MLflow à l'import.

Docker et Pipelines CI/CD

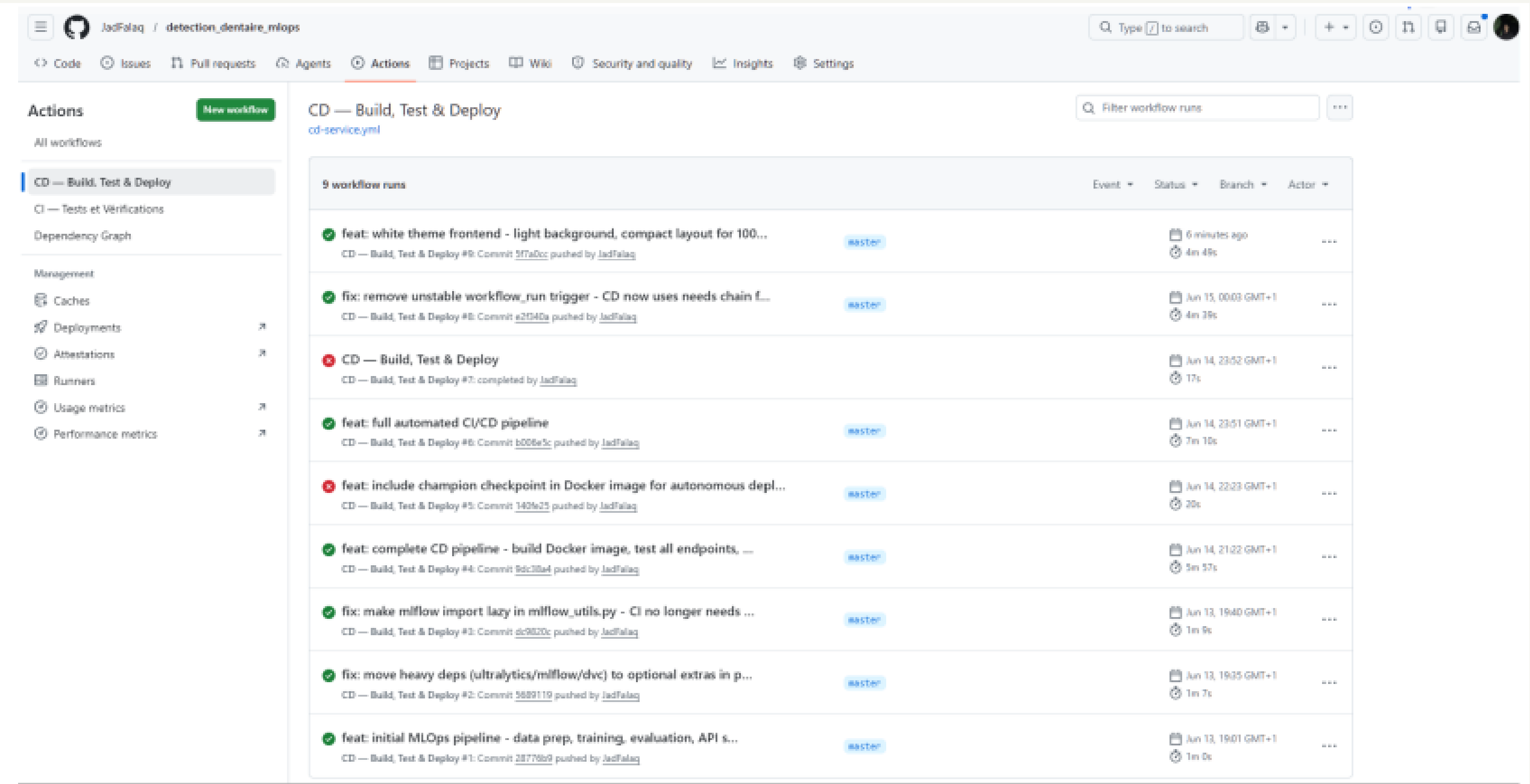
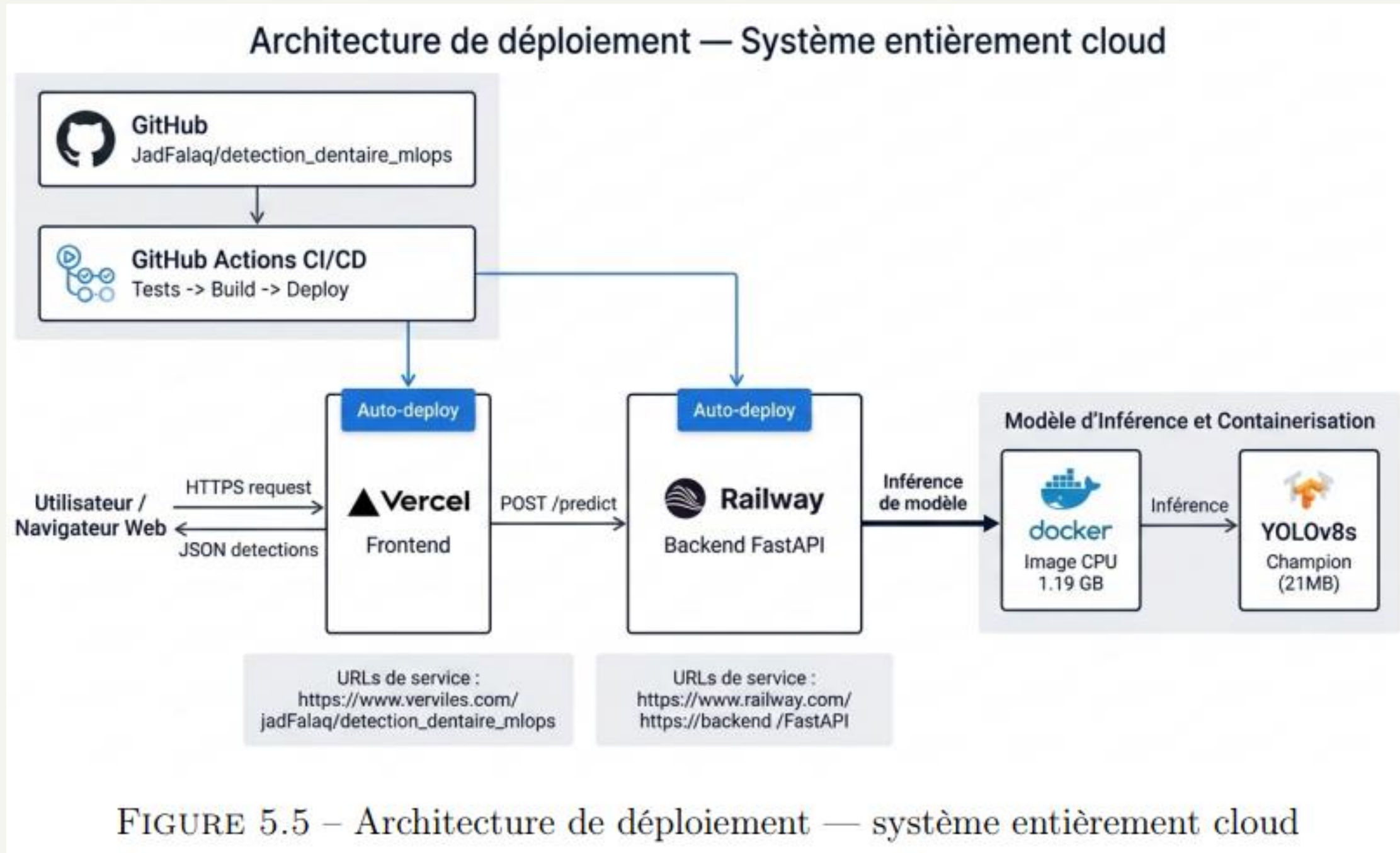


FIGURE 5.4 – Pipeline CD en action — tous les jobs au vert

Infrastructure Cloud Déployée

Architecture entièrement cloud



Bilan MLOps – Ce qui a été Accompli

Dimension	Ce qui a été fait	Outil	Statut
Données	Audit, nettoyage, remap 14→7 classes, pipeline reproductible	DVC	OK
Expérimentation	3 variantes entraînées, toutes tracées et comparées	MLflow	OK
Sélection	Registre Champion/Candidate/Archived, alias stable	YAML + CLI	OK
Service	API REST documentée, 4 endpoints, lazy loading	FastAPI	OK
Portabilité	Image Docker CPU 1.19 GB avec modèle intégré	Docker	OK
Automatisation	11 tests unitaires + 5 tests d'intégration + vérif. production	GitHub Actions	OK
Déploiement	Service cloud public, redéploiement automatique sur push	Railway + Vercel	OK

7/7

dimensions MLOps couvertes

100%

cloud, pas de machine locale

< 2 min

pipeline CI

1.19 GB

image Docker optimisée

16

tests automatisés

Ce qui a Fonctionné, Limites et Perspectives

Ce qui a bien fonctionné

- Penser le **déploiement** dès le début
- Checkpoint intégré dans l'**image Docker**
- **Registre de modèles** léger (YAML)
- **Tests d'intégration** dans le pipeline CD
- **Lazy loading** pour la réactivité
- Préprocessing **côté client**

Limites du système

- **mAP50 de 0.32** — performances modestes (corpus limité)
- **Pas de validation clinique** par des praticiens
- **Sensibilité** aux variations inter-cabinets (split external)
- Résolution contrainte à **832 px**

Perspectives d'évolution

- **Enrichir le corpus** (doubler la taille → +5-10 pts mAP50)
- Résolutions **1024-1280 px** avec GPU plus puissant
- **Tableau de bord** de monitoring (temps de réponse, requêtes)
- **MLflow Model Registry** complet
- **Validation clinique** par chirurgiens-dentistes

"L'entraînement de YOLOv8 a été la partie la plus simple. Ce qui a pris le plus de temps — et apporté le plus de valeur — c'est tout ce qui l'entoure : comprendre les données, tracer les expériences, concevoir un service qui fonctionne en production."

Merci

Questions ?

detection-dentaire-mlops.vercel.app

Falaq Jad — ENSIAS — MLOps 2025 / 2026

Encadrant : Mr Mohamed Lazzar

github.com/JadFalaq/detection_dentaire_mlops

AI Powered
Caries Detection